



Kampus
Merdeka
INDONESIA JAYA



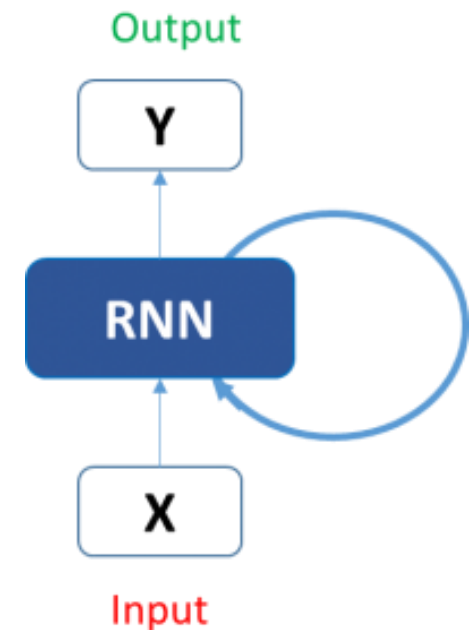
Recurrent Neural Network with Keras

Penyusun Modul: Chairul Aulia
Editor: Citra Chairunnisa



RNN

- Recurrent neural networks (RNNs) are one of the states of the art algorithm in deep learning especially good for sequential data (like text data, time series, financial, weather forecasting, etc.)
- The reason it became so popular is its internal memory.
- RNNs are the first deep learning algorithm that can remember the input.





Problems with RNN

- During back propagation, recurrent neural networks suffer from the **vanishing gradient problem**. This problem is when the gradient shrinks as it back propagates through time. If a gradient value becomes extremely small, it **doesn't contribute too much learning**.

new weight = weight - learning rate*gradient

2.0999 = 2.1

Not much of a difference

-

0.001

update value

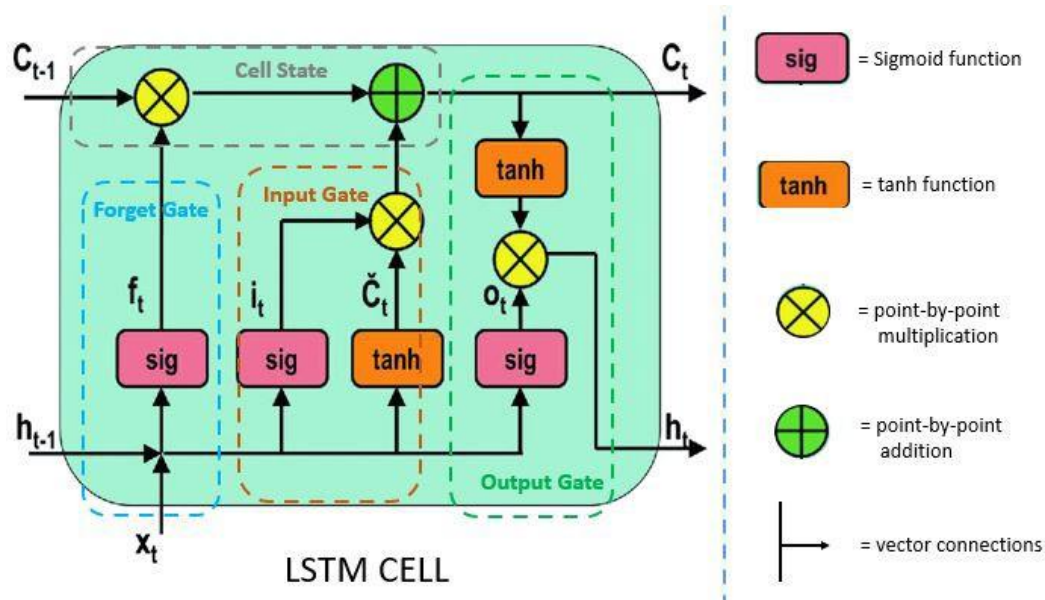
- Layers that get a small gradient update are usually the earlier layers. So, RNN's can forget what it seen in longer sequences, thus having a **short-term memory**.
- This is why RNNs suffer from short-term memory. If a sequence is long enough, they'll have a hard time carrying information from earlier time steps to later ones.



Solutions to short-term memory

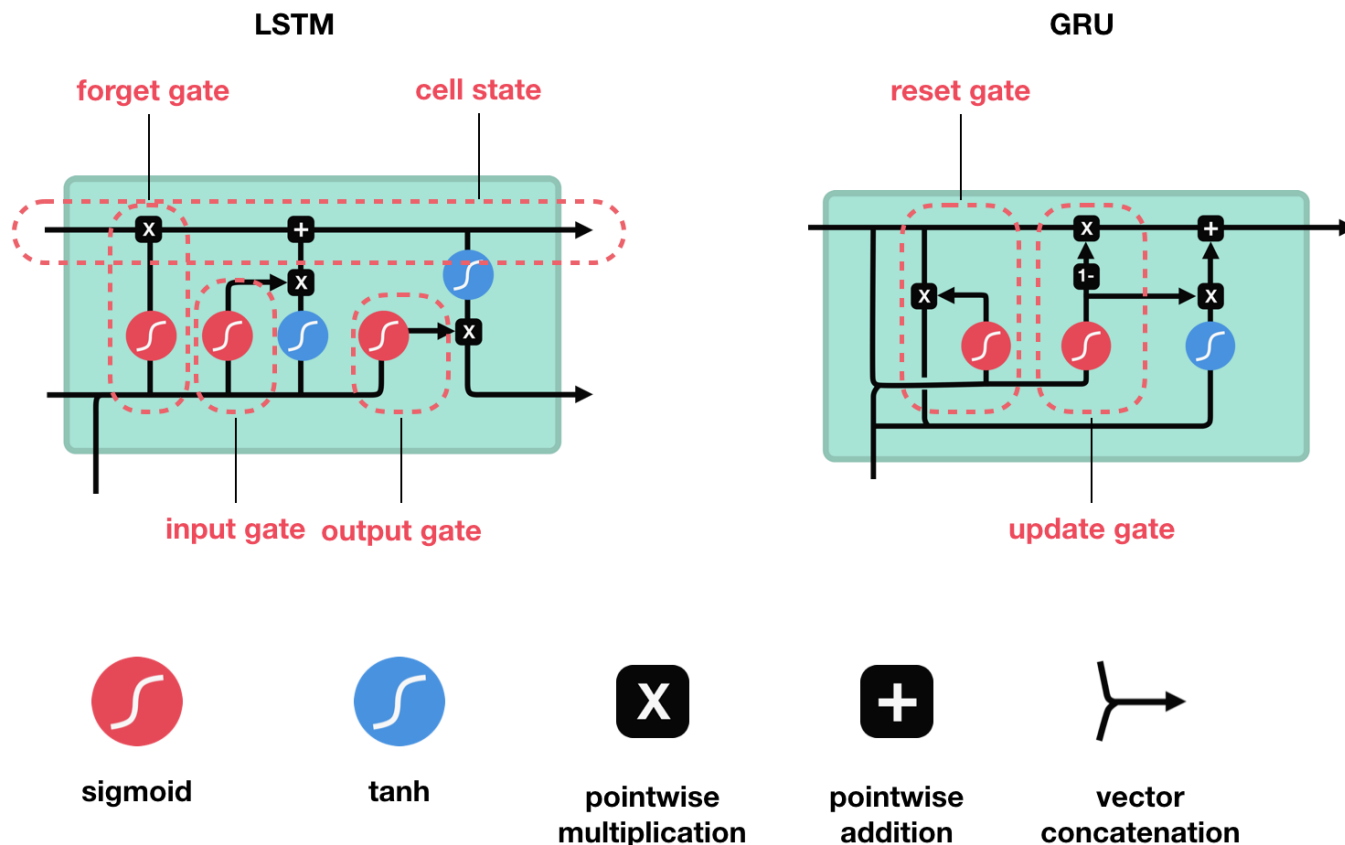
- **LSTM 's and GRU's** were created as the solution to short-term memory. They have internal mechanisms called **gates** that can **regulate the flow of information**.
- These gates can learn which data in a sequence is important to keep or throw away. By doing that, it can **pass relevant information** down the long chain of sequences to make predictions. Almost all state of the art results based on recurrent neural networks are achieved with these two networks

Long Short-Term Memory (LSTM)



- LSTM network basically contains three gates (forget, input, output) to control the flow of information and cells to hold information.
 - Forget gate decides how much information we will use from the previous cell state and how much we will 'throw away'.
 - The input gate decides which new information we are going to add to the cell state
 - The output gate decides what information we will pass to the network in the next instance of time
- The Cell States carries the information from initial to later time steps without getting vanished.

GRU (Gated Recurrent Unit)



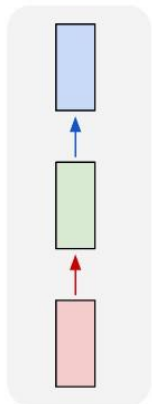
GRU has only two gates (update and reset gate).

- **Update gate** : A update gate is nothing but the combination of input and forget gate in a single unit .it decides what information to retain and what new information need to be added.
- **Reset gate** : It decides what information needs to be passed to the network in the next instance of time. It also merges the current cell state and hidden state, and makes some other changes.

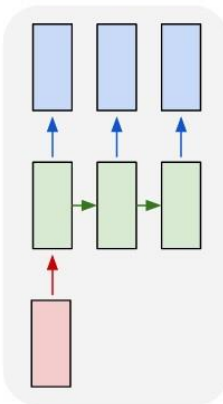


Types of RNN

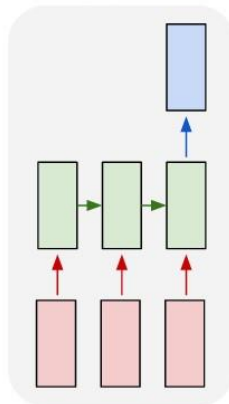
one to one



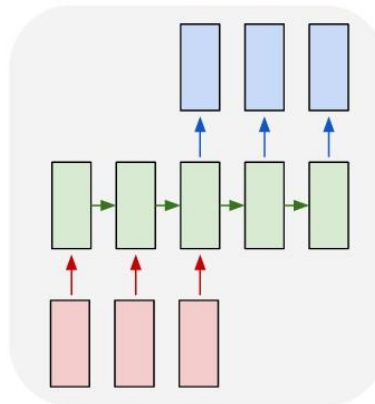
one to many



many to one



many to many



The common ways to use are 'many to one' and 'many to many' mode

Example for implementing many to one:

```
model = Sequential()
model.add(SimpleRNN(units, input_shape=(timesteps, data_dim)))
```

For many to many:

```
model = Sequential()
model.add(SimpleRNN(units, input_shape=(timesteps, data_dim), return_sequences=True))
```

Units = positive integer, dimensionality of the output space

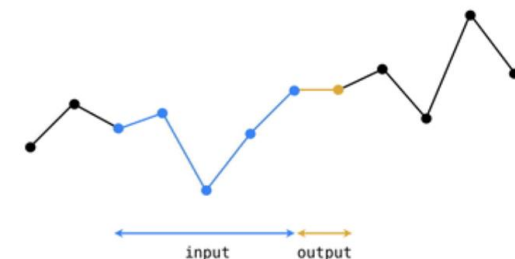
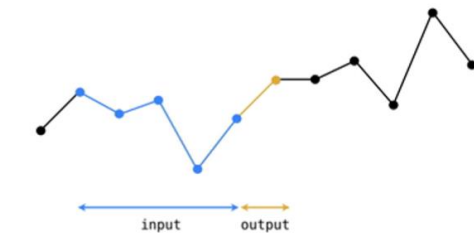
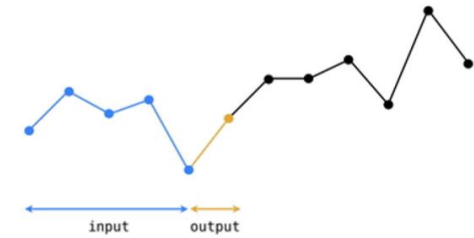


Predicting Time Series Data

In order to apply our RNN we treat the time series prediction problem as a regression problem, and so need to use a sliding window to construct a set of associated input/output pairs to regress on. Look at the picture on the side to understand the process (this is an example of many to one).

Input	Output
$\langle s_1, s_2, s_3, s_4, s_5 \rangle$	s_6
$\langle s_2, s_3, s_4, s_5, s_6 \rangle$	s_7
$\vdots$	$\vdots$
$\langle s_{P-5}, s_{P-4}, s_{P-3}, s_{P-2}, s_{P-1} \rangle$	s_P

Set of input/output pairs produced shown in the table above is our set of training input and output. In real practice, you could change the size of your input-output and the timesteps (the steps between each row) to find the optimum forecasting. Unless you already specify the problem, for example you want to predict a value based on the past 7 days data, so the input size needs to be 7 days length.





Some Applications of RNN

RNNs are very good at making predictions, especially when our data is sequential:

- **Stock market forecasts.** The values of a share depend largely on the values it had previously
- **Sequence generation.** As long as data are sequences and data in an instant t depends on the data in the instant $t-1$.
- **Text generation.** For example, when your cell phone suggests words. It looks at the last word you have written, and at the letters, you are writing at that moment to suggest the next letters or even words.
- **Voice recognition.** In this case, we have the previous word recognized, and the audio that reaches us at that moment.
- **Weather forecasts.**



Exercise

Let's implement RNN model to predict the apple prices and IBM stock price

Use **normalized_apple_prices** and **IBM stock price** datasets